

Computer Programming Summer Interest Group

Session 14 / Generators

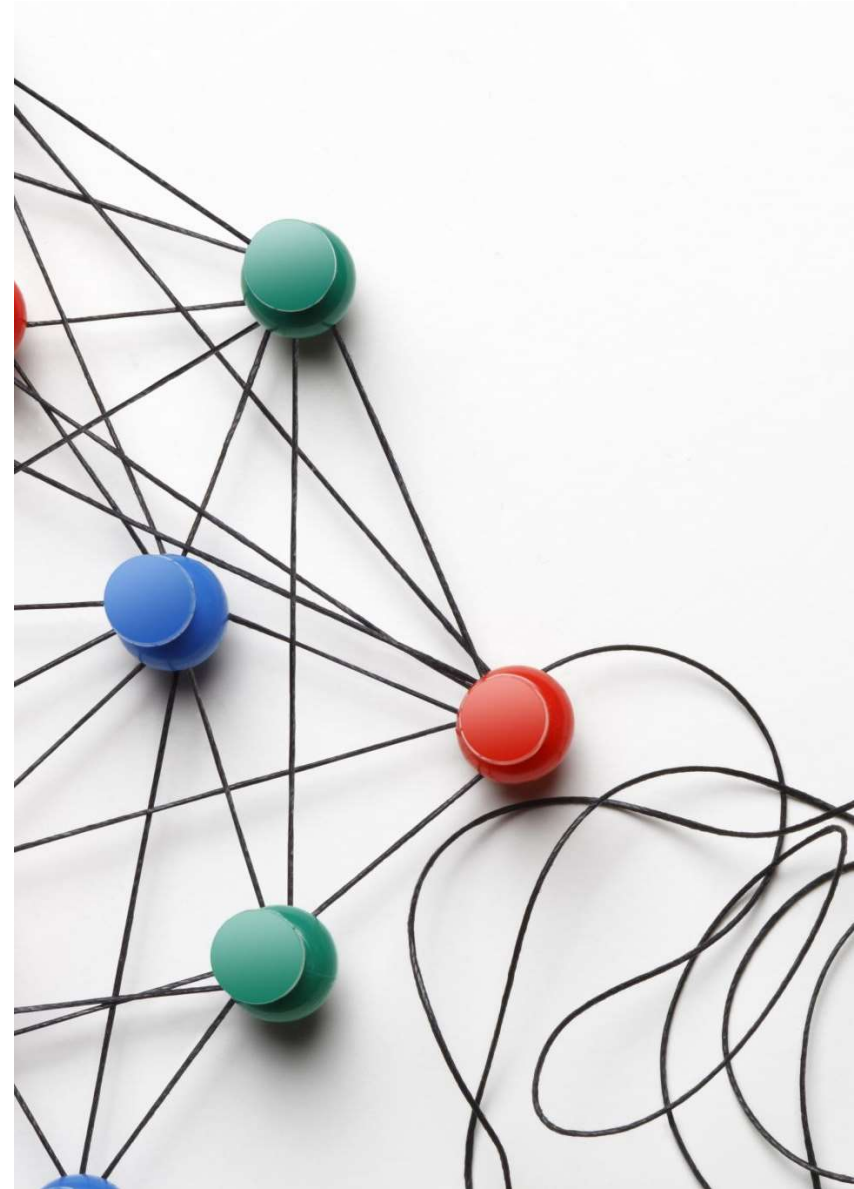
Hebrews 13:5-6

Keep your life free from love of money, and be content with what you have, for he has said, “I will never leave you nor forsake you.” So we can confidently say,

“The Lord is my helper;
I will not fear;
what can man do to me?”

Objectives

- By the end of this session, you will be able to:
 - Understand what generators are and how they differ from lists and iterators.
 - Learn how to create a generator using functions and the 'yield' keyword.
 - Use generator expressions for concise generator creation.
 - Recognize the advantages of generators in terms of memory efficiency and performance.
 - Practice writing and using generators in real-world scenarios.





Generators

Generators allow python programs to operate on large datasets or value ranges without storing everything in memory

This permits processing content “just-in-time” while remaining memory efficient

Generators come in two flavors

- Generator functions – Functions that can be used to generate the next value
- Generator expression – A shorthand notation, similar to list comprehensions, to define simple generators
 - In lesson 12, we used generator expressions to mimic the ability to perform tuple comprehension

The 'yield' keyword

- The 'yield' keyword can be thought of as temporary return.
 - It returns a value, but the state of the generator function is not lost and can be resumed from where it yielded
- Something using the generator calls the function and control switches between the generator function being called and the function calling the generator
- The "my_range" function to the right is a generator function
- Each iteration of the while loop 'yield's the next value in the range.

```
def my_range(start: int,
              stop: int,
              step: int = 1):
    assert step > 0
    value = start
    while value < stop:
        yield value
        value += step

print("Values in range 0..10")
for value in my_range(0, 10):
    print(value)

print("Even values in range 0..10")
for value in my_range(0, 10, 2):
    print(value)
```

The 'next' function

- The 'next' function runs a generator until the next 'yield' occurs and gets the value yielded

```
def my_range(start: int,
             stop: int,
             step: int = 1):
    assert step > 0
    value = start
    while value < stop:
        yield value
        value += step

gen_0 = my_range(0, 10)
print("Values in range 0..10")
for _ in range(10):
    print(next(gen_0))
```


The 'StopIteration' exception

- In many cases, we may not know when to stop incrementing the generator.
- If we try to generate values in excess of those available, python will raise a "StopIteration" exception
- This can be caught and used to stop further iterations without halting the program

```
def my_range(start: int,
             stop: int,
             step: int = 1):
    assert step > 0
    value = start
    while value < stop:
        yield value
        value += step

gen_0 = my_range(0, 10)
print("Values in range 0..10")
while True:
    try:
        print(next(gen_0))
    except StopIteration:
        break
```

A slightly more complicated example

- This generator produces a sequence of Fibonacci number
- We can use the enumerate function to get the iteration index along with the corresponding Fibonacci value
- State inside 'fib' is maintained between 'yield' expressions.
- Uses a Python trick to “simultaneously” assign values to u and v inside the loop in 'fib'

```
def fib(count: int):  
    u, v = 0, 1  
    yield u  
    for _ in range(1, count):  
        u, v = v, u + v  
        yield u  
  
for idx, n in enumerate(fib(20)):  
    print(f"{idx}: {n}")
```


Generator Expressions

- We have seen these already in session 12
 - Used them to mimic a tuple comprehension (which is not in python)
- This is like comprehensions but yields each element until all are consumed.
- Useful for defining simple generators
- Syntax is very similar to comprehension

```
N = 11

def cubes(max_n: int):
    for n in range(max_n):
        yield n**3

# Generator expression
ge = (n**3 for n in range(N))

print("Use of generator expression:")
for n in ge:
    print(n)

print("Use of function which acts as a generator:")
for n in cubes(N):
    print(n)
```



Activities

- Consider the equation:

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}$$

- Here the $n! = 1 \times 2 \times 3 \times \dots \times n$ (i.e. n factorial, with $0! = 1$) and the Σ represents the summation of the elements for $n = 0$ to infinity
- This is known as the [Taylor Series expansion of \$e^x\$](#) .
- Use a generator and the reduce function (introduced in session 13) to approximate e^x
- How many iterations do we need to come close to the real value (use `math.exp` to compare with the built in function in python).